Лабораторная работа №2.
Построение грамматики для формального языка DSL и реализация синтаксического анализатора.

Берлет Максим гр. 3384

Содержание

1 Постановка задачи **3**

2 Описание решения **4**

2.1 Описание языка 4

2.2 Описание LL1 грамматики 4

2.3 Описание парсера 5

2.4 Визуализация 5

3 Демонстрация **6**

3.1 Корректные предложения 6

3.2 Некорректные предложения 10

4 Выводы **11**

1 Постановка задачи

Условие задания

Разработать формальную грамматику для предметно-ориентированного языка (DSL) запросов к базе данных и реализовать синтаксический анализатор, строящий деревья разбора для корректных выражений.

Входные данные

Текстовый файл, содержащий список запросов.
Запросы могут быть как синтаксически правильными, так и содержать ошибки.

Выходные данные

Для каждого запроса программа должна вывести:

1. Исходный запрос
2. Результат синтаксического анализа (успех/неудача)
3. В случае успеха — дерево разбора в текстовом или графическом виде
4. В случае неудачи — диагностику ошибки (на каком токене произошла ошибка)

2 Описание решения

2.1 Описание языка

По условию нужно придумать LL1 грамматику описывающую язык запросов к БД на русском языке, по которой можно построить синтаксический анализатор (парсер). Синтаксически корректные запросы будут обработаны и в ходе парсинга построено дерево разбора. Некорректные запросы не будут распознаны и будет показано, где парсер не смог распознать символ.

Типичный запрос к БД - это запрос SELECT. Именно его и будем описывать в грамматике. У оператора SELECT есть возможность выбрать все колонки, либо какие-то конкретные, также можно указать из какой таблицы выбирать, условия для отбора записей. Кроме того, запрос можно сортировать по колонкам и выводить не все записи, а ограниченное их количество.

2.2 Описание LL1 грамматики

Опишем правила, задающие LL1 грамматику.

```
<root> ::= <query>
<query> ::= <select-operator><columns><source><conditions><sort-operator><limit-operator>
<select-operator> ::= "выбрать" | "достать" | "выдать" | "взять"
<columns> ::= <all> | <columns-list>
<all> ::= "всё" | "все"
<columns-list> ::= <column><columns-list-tail>
<column> ::= <существительное>
<columns-list-tail> ::= <comma><column><columns-list-tail> | ε
<comma> ::= ","
<source> ::= <from-table><существительное>
<from-table> ::= <from><table>
<from> ::= "из"
<table> ::= "таблицы"
<conditions> ::= <where><condition>|ε
<where> ::= "где"
<condition> ::= <expression><condition-tail>
<condition-tail> ::= <chain><expression>|ε
<expression> ::= <expression-head><inequality-sings><condition-word>
<expression-head> ::= <существительное>  | <имя прилагательное>
<chain> ::= "и" | "или"
<inequality-sings> ::= "<" | ">" | "==" | "<=" | ">=" | "! ="| <компаратив>
<condition-word> ::= <существительное>  | <число>  | <наречие>  | <имя прилагательное>
<sort-operator> ::= <sort><предлог><column><предлог><order> | ε
<sort> ::= "сортировать"
<order> ::= "возрастанию"|"убыванию"|ε
<limit-operator> ::= <limit><целое число>  | ε
<limit> ::= "ограничить"
```

2.3 Описание парсера

В качестве парсера был реализован классический рекурсивный анализатор, где каждому правилу соответствует функция. Нетерминальные правила проваливаются глубже в другие правила через соответствующие функции, а функции терминальных правил, возвращают узлы дерева разбора, в которых хранятся символы(токены). Для определения части речей речи слов была использована библиотека *pymorphy3*.

Изначально на вход поступает запрос - строка, строка разбивается на слова и пунктуацию(запятые) - это называется токенами, далее класс парсер получает на вход список токенов и начинает синтаксический разбор. Если на каком-то шаге парсер получил токен, который он не ожидал получить, он бросает исключение SyntaxError, в сообщении которого пишется, что именно и где именно пошло не так.

2.4 Визуализация

Во время синтаксического разбора происходит построение дерева. При проходе через каждое правило создается новая вершина, которая соединяется ребром с правилом, из которого была вызвана функция. Для визуализации был использован модуль *graphviz* для python.

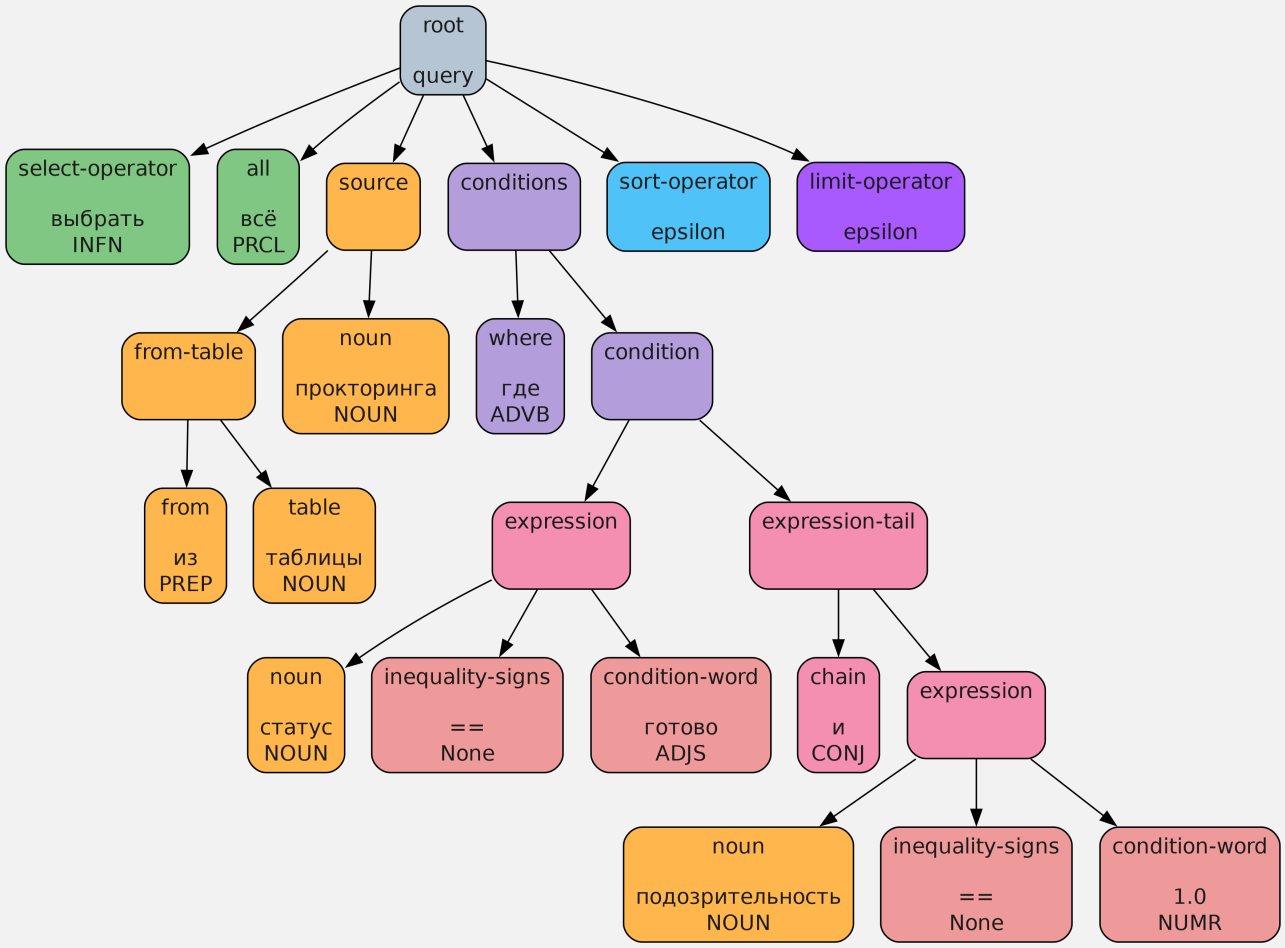
3 Демонстрация

3.1 Корректные предложения

В папке *examples* лежит текстовый файл *correct.txt*, в котором записаны примеры корректных запросов.

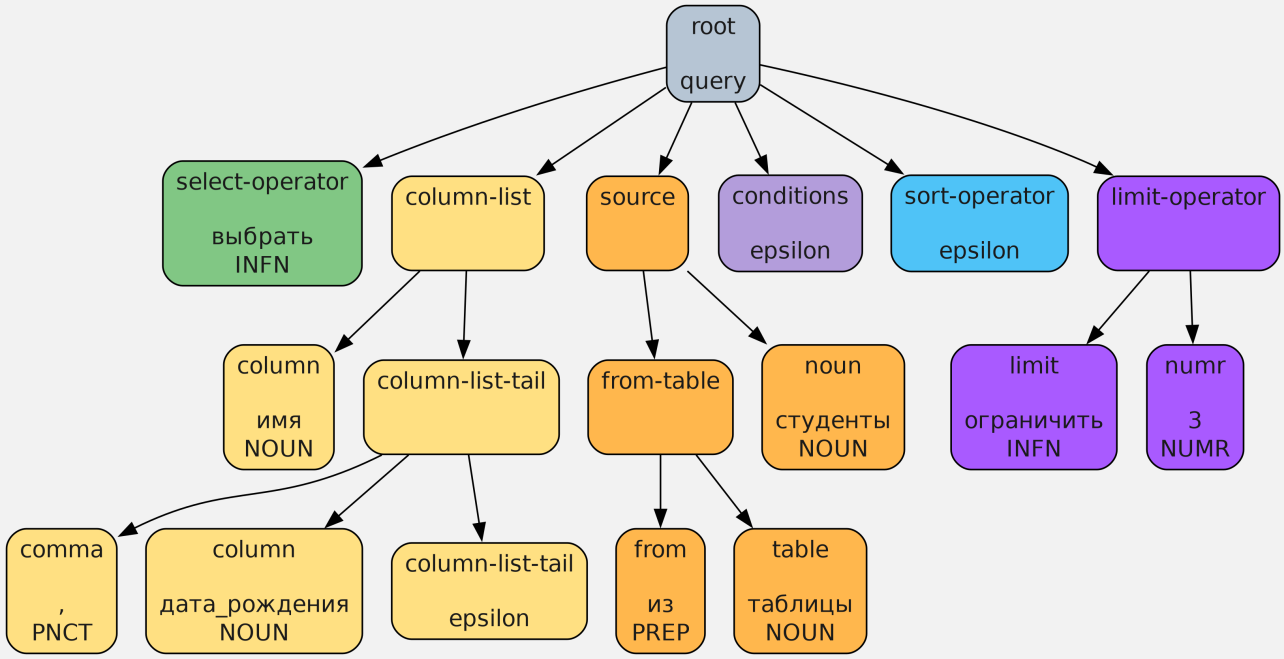
Пример 1

выбрать всё из таблицы прокторинга где статус == готово и подозрительность == 1.0



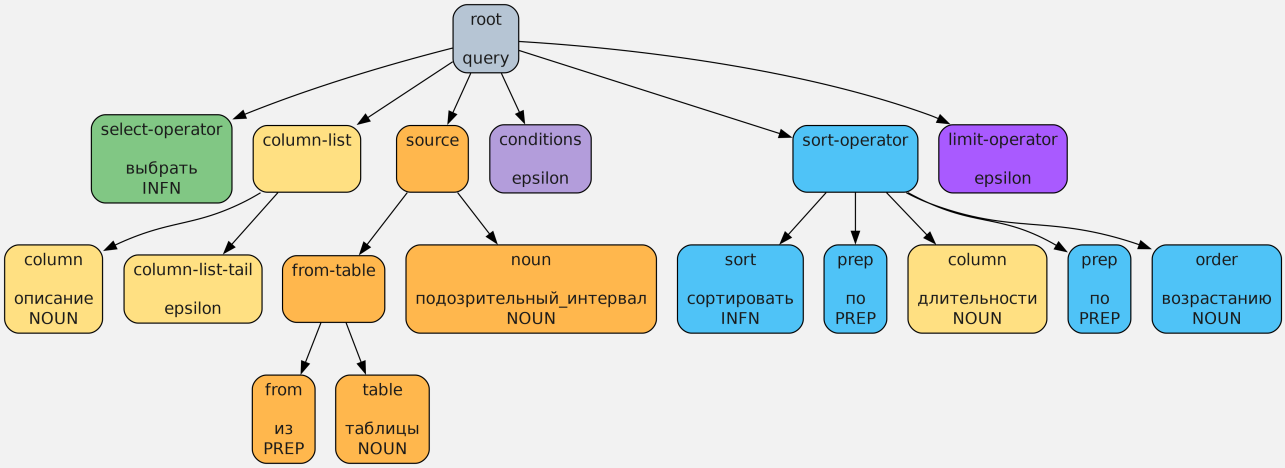
Пример 2

выбрать имя, дата_рождения из таблицы студенты ограничить 3



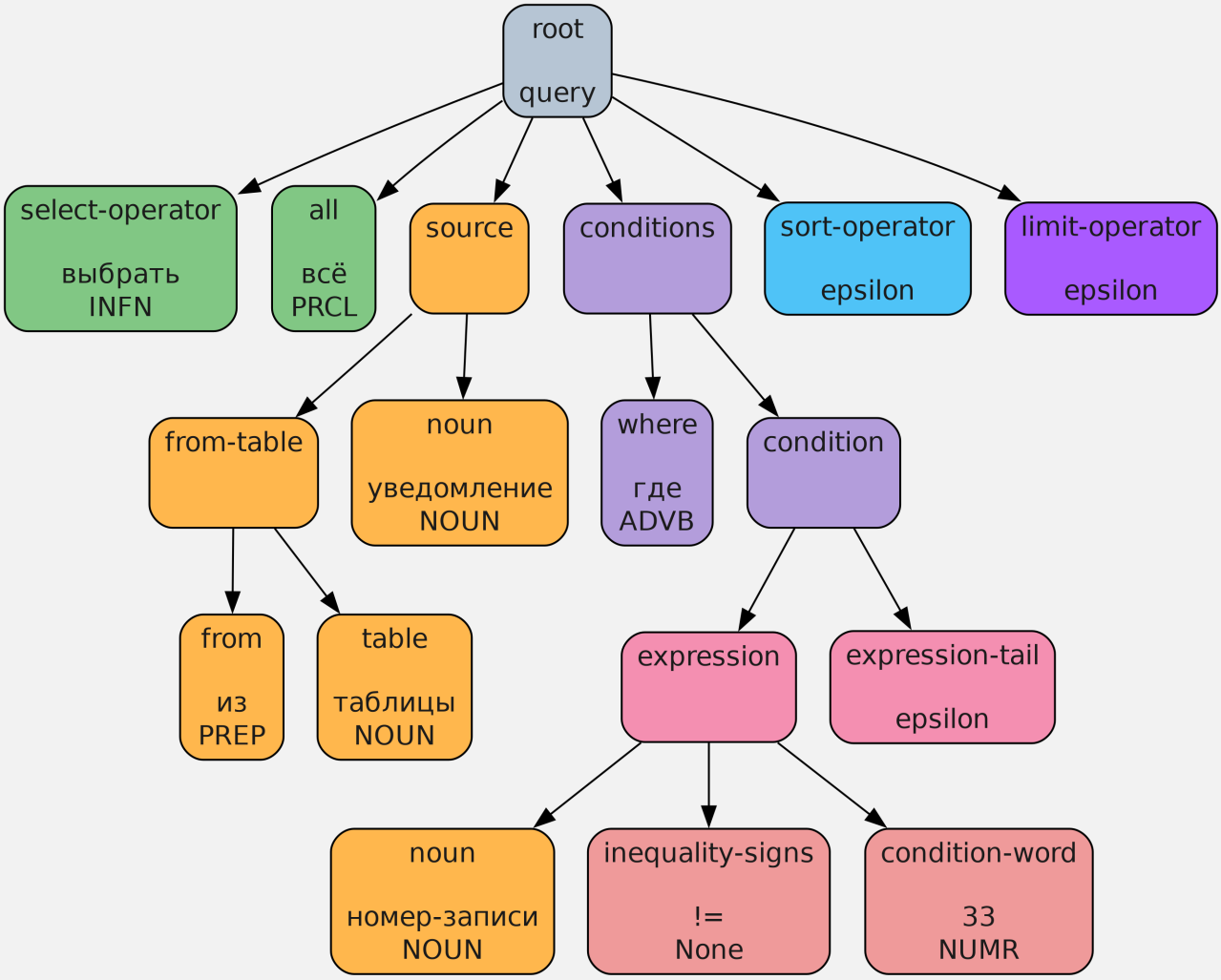
Пример 3

выбрать описание из таблицы подозрительный_интервал сортировать по длительности по возрастанию



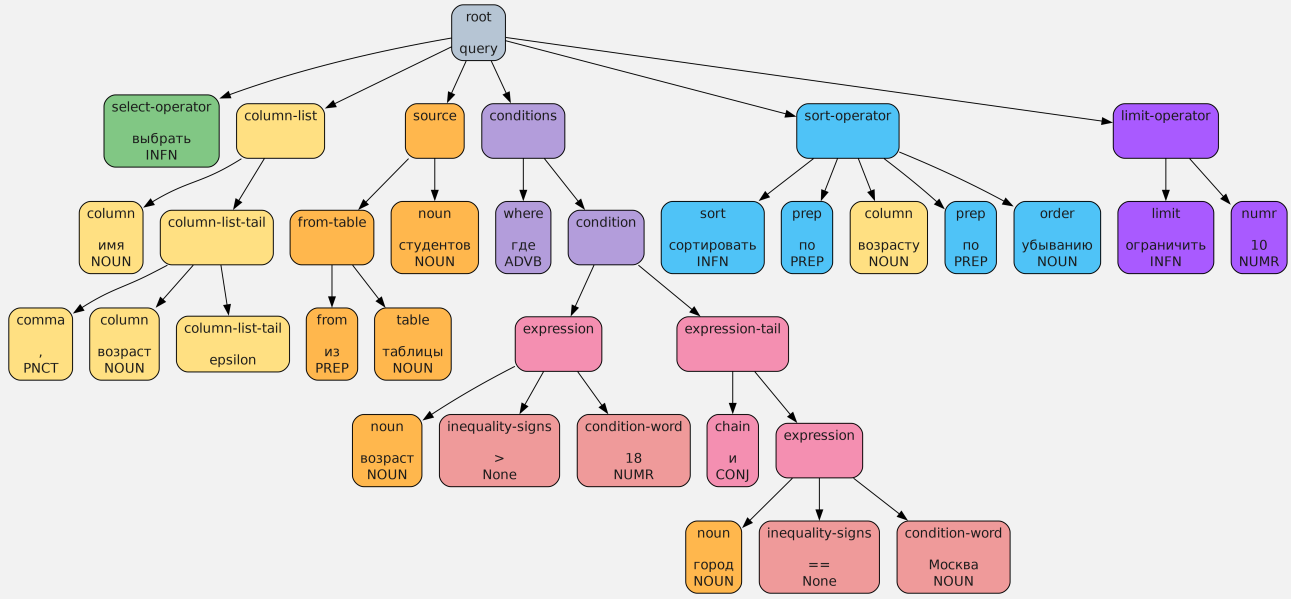
Пример 4

выбрать всё из таблицы уведомление где номер-записи ! = 33



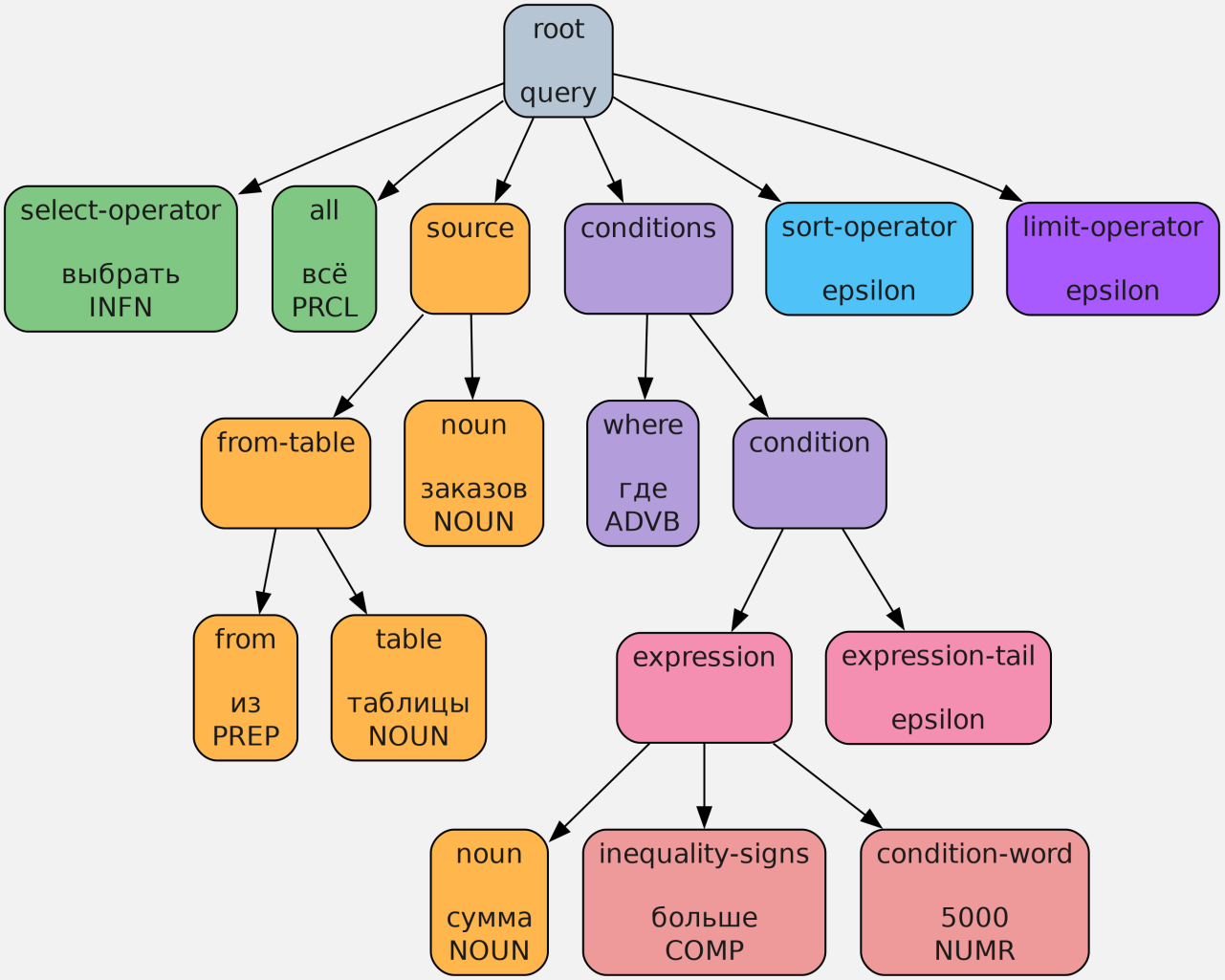
Пример 5

выбрать имя, возраст из таблицы студентов где возраст > 18 и город == Москва сортировать по возрасту по убыванию ограничить 10



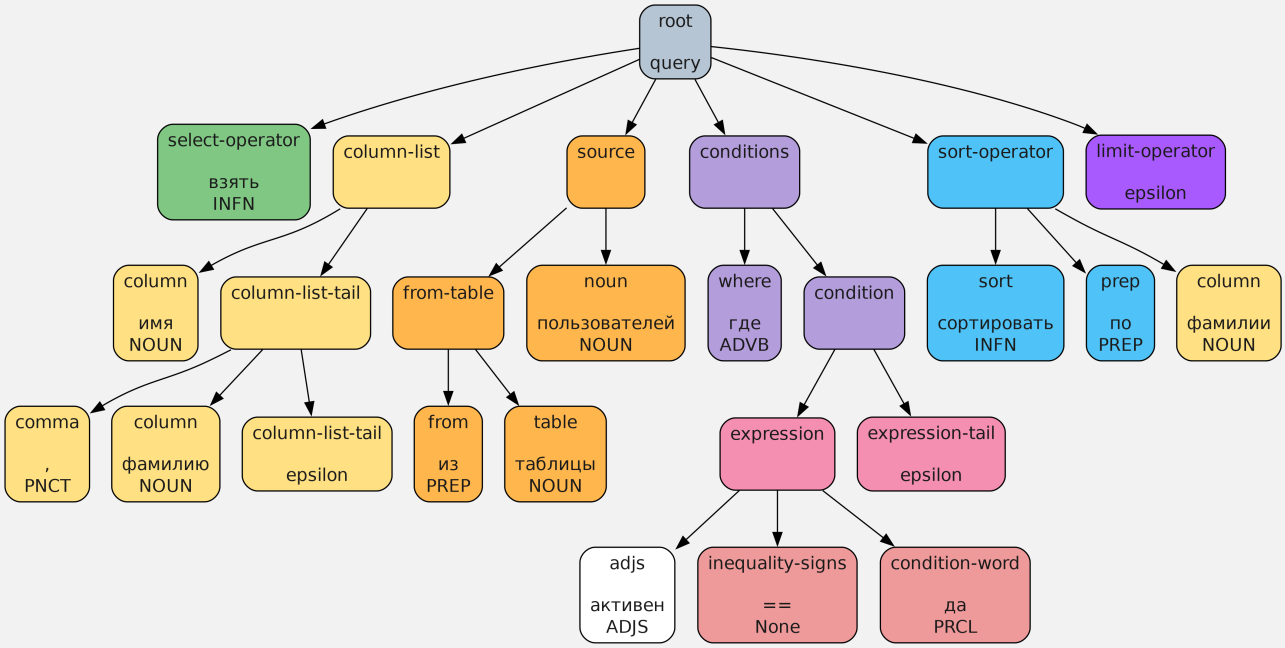
Пример 6

выбрать всё из таблицы заказов где сумма больше 5000



Пример 7

взять имя, фамилию из таблицы пользователей где активен == да сортировать по фамилии



3.2 Некорректные предложения

В папке *examples* лежит текстовый файл *wrong.txt*, в котором записаны примеры некорректных запросов.

Пример 1

выбрать имя из таблицы пользователи ограничить много

```
выбрать имя из таблицы пользователи ограничить много
                                                 ^^^^^
Expected NUMR got много with pos ADVB.
```

Пример 2

выбрать имя возраст из таблицы студентов сортировать имя по убыванию

```
выбрать имя из таблицы пользователи где возраст ~= 20
                                                ^^
Expected ['<', '>', '==', '!=', '>=', '<='] token got ~= token.
```

Пример 3

достать название цену из таблицы товаров где цена <= тысяча

```
выбрать быстро из таблицы пользователи
        ^^^^^^
Expected NOUN token got быстро token with pos ADVB.
```

4 Выводы

В ходе выполнения лабораторной работы была разработана LL1 грамматика, описывающая запросы к БД на русском языке. По грамматике был реализован синтаксический анализатор(парсер), рекурсивно обрабатывающий строку с запросом. Были построены и визуализированы деревья синтаксического разбора. В файлах *correct.txt* и *wrong.txt* приведены примеры корректных и некорректных запросов.

Исходный код